06. 타원곡선 암호

목차

1. 개요
2. halo2-ecc 모듈 구조
3. EcPoint 타입
4. 타원곡선 기본 연산
5. 스칼라 곱셈 및 MSM
6. 필드 연산 최적화
7. 관련 문서

개요

halo2-ecc는 halo2-lib의 일부로, 영지식증명 회로 내에서 타원곡선 암호(Elliptic Curve Cryptography, ECC) 연산을 효율적으로 구현하기 위한 라이브러리입니다. 이 모듈은 다음과 같은 핵심 기능을 제공합니다:

- **임의의 타원곡선 지원**: Short Weierstrass 형태의 타원곡선 ($y^2 = x^3 + b$)
- **효율적인 필드 연산**: 큰 정수 연산과 소수체(prime field) 연산 최적화
- **타원곡선 점 연산**: 점 덧셈, 배가, 스칼라 곱셈
- **고급 알고리즘**: MSM(Multi-Scalar Multiplication), Pippenger 알고리즘
- **특정 곡선 최적화**: secp256k1, BN254 등 주요 곡선에 대한 특화 구현

halo2-ecc 모듈 구조

halo2-ecc는 계층적 구조로 설계되어 있으며, 각 모듈은 특정 책임을 담당합니다.

전체 디렉토리 구조

```
halo2-ecc/
├── src/
| ├── lib.rs # 모듈 진입점
| ├── bigint/ # 큰 정수 연산
| ├── fields/ # 소수체 연산
| ├── ecc/ # 타원곡선 점 연산
| ├── secp256k1/ # secp256k1 곡선 특화
| ├── bn254/ # BN254 곡선 특화
| └── grumpkin/ # Grumpkin 곡선 특화
├── benches/ # 벤치마크
└── Cargo.toml
```

모듈별 역할

1. bigint 모듈

위치: `halo2-ecc/src/bigint/`

역할: 영지식증명 회로 내에서 큰 정수(big integer) 연산을 구현합니다. 네이티브 필드보다 큰 정수를 다루기 위해 limb 기반 표현을 사용합니다.

주요 기능:

- **CRTInteger**: Chinese Remainder Theorem 표현을 사용한 큰 정수
- **OverflowInteger**: carry를 허용하는 중간 계산용 정수

- **ProperCrtUint**: 올바르게 범위 검사된 큰 정수
- 기본 연산: 덧셈, 뺄셈, 곱셈 (carry 처리 포함)

핵심 개념:

```
// limb 기반 표현: 큰 정수를 여러 작은 조각(limb)으로 분할
// 예: 256비트 정수를 88비트 limb 3개로 표현
// value = limb[0] + limb[1] * 2^88 + limb[2] * 2^176
```

2. fields 모듈

위치: `halo2-ecc/src/fields/`

역할: 소수체(prime field) 연산을 구현합니다. 타원곡선의 기저 필드(base field)와 스칼라 필드(scalar field)에서의 연산을 지원합니다.

주요 컴포넌트:

- **FpChip**: 소수체 Fp의 연산 칩
- **Fp2Chip**: 확장 필드 Fp^2 연산 (복소수 형태)
- **Fp12Chip**: 확장 필드 Fp^{12} 연산 (pairing 연산용)
- **FieldChip trait**: 모든 필드 칩이 구현해야 하는 공통 인터페이스

핵심 메서드:

```
pub trait FieldChip<F: BigPrimeField> {
// carry 없는 연산 (중간 계산용)
fn mul_no_carry(&self, ctx: &mut Context<F>, a, b) -> UnsafeFieldPoint;
fn add_no_carry(&self, ctx: &mut Context<F>, a, b) -> UnsafeFieldPoint;
// carry 및 modular reduction 수행
```

```
fn carry_mod(&self, ctx: &mut Context<F>, a: UnsafeFieldPoint) -> FieldPoint;

// 나눗셈 (역원 계산 포함)

fn divide_unsafe(&self, ctx: &mut Context<F>, a, b) -> FieldPoint;

}
```

3. ecc 모듈

위치: `halo2-ecc/src/ecc/`

역할: 타원곡선 점 연산의 핵심 구현을 제공합니다. 임의의 Short Weierstrass 곡선에 대해 작동합니다.

주요 서브모듈:

- **mod.rs**: 기본 점 연산 (덧셈, 배가, 스칼라 곱셈)
- **ecdsa.rs**: ECDSA 서명 검증
- **fixed_base.rs**: 고정 기저점 스칼라 곱셈 최적화
- **pippenger.rs**: Pippenger 알고리즘 기반 MSM
- **schnorr_signature.rs**: Schnorr 서명 검증

핵심 타입:

```
// 타원곡선 점 표현

pub struct EcPoint<F: BigPrimeField, FieldPoint> {

pub x: FieldPoint,

pub y: FieldPoint,

}


// ECC 연산 칩
```

```
pub struct EccChip<'chip, F: BigPrimeField, FC: FieldChip<F>> {
pub field_chip: &'chip FC,
}
```

4. secp256k1 모듈

위치: `halo2-ecc/src/secp256k1/`

역할: Bitcoin과 Ethereum에서 사용되는 secp256k1 곡선에 특화된 구현을 제공합니다.

특징:

- secp256k1 곡선 파라미터 사전 정의
- ECDSA 서명 검증 최적화
- 벤치마크 및 테스트 포함

곡선 파라미터:

```
p = 2^256 - 2^32 - 977 (기저 필드 모듈러스)
n = FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141 (곡선 차수)
G = (generator point)
```

5. bn254 모듈

위치: `halo2-ecc/src/bn254/`

역할: BN254 (Barreto-Naehrig) 곡선에 특화된 구현을 제공합니다. 특히 pairing 연산을 지원합니다.

주요 기능:

- **pairing.rs**: Optimal Ate pairing 구현
- **final_exp.rs**: Final exponentiation 최적화

- G1, G2 점 연산
- Fp12 확장 필드 연산

Pairing 연산:

```
// e: G1 × G2 → GT
// e(P, Q) 계산
pub fn pairing<F>(
chip: &PairingChip<F>,
ctx: &mut Context<F>,
Q: EcPoint<F, Fp2Point>, // G2 점
P: EcPoint<F, FpPoint>, // G1 점
) -> Fp12Point;
```

모듈 간 의존성

```
┌─────────────┐
| bigint | ← 가장 낮은 레벨: 큰 정수 연산
└─────┬───────┘
|
↓
┌─────────────┐
| fields | ← 소수체 연산 (bigint 사용)
└─────┬───────┘
|
```

```
↓

┌──────────────┐

| ecc | ← 타원곡선 점 연산 (fields 사용)

└──────┬───────┘

|

↓

┌──────────────┬──────────────┐

| secp256k1 | bn254 | ← 특정 곡선 구현 (ecc 사용)

└──────────────┴──────────────┘
```

Cargo.toml 구성

```
[package]

name = "halo2-ecc"

version = "0.4.1"



[dependencies]

halo2-base = { version = "=0.4.1", path = "../halo2-base" }

itertools = "0.11"

num-bigint = { version = "0.4", features = ["rand"] }

num-integer = "0.1"

num-traits = "0.2"

rand = "0.8"
```

```
rand_chacha = "0.3.1"

rayon = "1.8" # 병렬 처리용


[features]

default = ["jemallocator", "halo2-axiom"]

# 메모리 할당자 선택

jemallocator = ["halo2-base/jemallocator"]

mimalloc = ["halo2-base/mimalloc"]
```

주요 의존성:

- **halo2-base**: 기본 회로 구성 요소
- **num-bigint**: 큰 정수 연산 라이브러리
- **rayon**: 병렬 처리 (MSM 최적화용)
- **rand_chacha**: 결정론적 난수 생성

사용 예시

```
use halo2_base::gates::RangeChip;

use halo2_ecc::fields::fp::FpChip;

use halo2_ecc::ecc::{EccChip, EcPoint};

use halo2_ecc::secp256k1::Secp256k1Affine;


// 1. RangeChip 생성 (halo2-base)

let  range_chip = RangeChip::default(lookup_bits);
```

```
// 2. FpChip 생성 (fields 모듈)
let  fp_chip = FpChip::<F, Fq>::new(
&range_chip,
limb_bits, // 예: 88
num_limbs, // 예: 3
);


// 3. EccChip 생성 (ecc 모듈)
let  ecc_chip = EccChip::new(&fp_chip);


// 4. 타원곡선 점 로드
let P = ecc_chip.load_private::<Secp256k1Affine>(ctx, (x, y));


// 5. 점 연산 수행
let Q = ecc_chip.double(ctx, P);
let R = ecc_chip.add_unequal(ctx, P, Q, true);
```

관련 문서

- 02. 핵심 개념 - Context, AssignedValue 기본 개념
- 04. 제약 조건 시스템 - FlexGate, RangeGate
- 05. 가상 영역 관리 - 회로 최적화

- 07. 실전 예제 분석 - ECDSA, Pairing 구현

참고 자료

- 소스 파일: `halo2-ecc/src/lib.rs`
- 소스 파일: `halo2-ecc/Cargo.toml`
- 소스 파일: `halo2-ecc/src/ecc/mod.rs`
- 소스 파일: `halo2-ecc/src/fields/mod.rs`
- 소스 파일: `halo2-ecc/src/bigint/`

EcPoint 타입

개요

`EcPoint` 는 halo2-ecc에서 타원곡선 위의 점을 표현하는 핵심 타입입니다. 이 타입은 제네릭으로 설계되어 다양한 필드 표현과 타원곡선에 대해 작동할 수 있습니다.

EcPoint 구조체

소스: `halo2-ecc/src/ecc/mod.rs`

```
#[derive(Debug)]
pub struct EcPoint<F: BigPrimeField, FieldPoint> {
pub x: FieldPoint,
pub y: FieldPoint,
_marker: PhantomData<F>,
}
```

필드 설명:

- **x**: 점의 x 좌표 (필드 원소)
- **y**: 점의 y 좌표 (필드 원소)
- **_marker**: 제네릭 타입 `F` 를 위한 PhantomData (컴파일 타임 타입 정보)

제네릭 파라미터:

- **F**: 네이티브 필드 타입 (회로의 기본 필드, 예: `Fr`)
- **FieldPoint**: 필드 원소의 표현 방식 (예: `ProperCrtUint<F>`)

주요 메서드

```
impl<F: BigPrimeField, FieldPoint> EcPoint<F, FieldPoint> {
// 새로운 점 생성
pub fn new(x: FieldPoint, y: FieldPoint) -> Self {
Self { x, y, _marker: PhantomData }
}

// x 좌표 참조
pub fn x(&self) -> &FieldPoint {
&self.x
}

// y 좌표 참조
pub fn y(&self) -> &FieldPoint {
&self.y
```

```
}

}
```

Clone 구현

```
impl<F: BigPrimeField, FieldPoint: Clone> Clone for EcPoint<F, FieldPoint> {

fn  clone(&self) -> Self {

Self {

x: self.x.clone(),

y: self.y.clone(),

_marker: PhantomData

}

}

}


// 참조에서 소유 타입으로 변환 (가독성 향상)

impl<'a, F: BigPrimeField, FieldPoint: Clone> From<&'a EcPoint<F, FieldPoint>>

for EcPoint<F, FieldPoint>

{

fn  from(value: &'a EcPoint<F, FieldPoint>) -> Self {

value.clone()

}
```

```
}
```

StrictEcPoint 구조체

`StrictEcPoint` 는 x 좌표가 이미 reduced form(축약된 형태)으로 제약된 점을 나타냅니다. 이를 통해 x 좌표 비교를 더 효율적으로 수행할 수 있습니다.

```
#[derive(Clone, Debug)]
pub struct StrictEcPoint<F: BigPrimeField, FC: FieldChip<F>> {
pub x: FC::ReducedFieldPoint, // 축약된 x 좌표
pub y: FC::FieldPoint, // 일반 y 좌표
_marker: PhantomData<F>,
}

impl<F: BigPrimeField, FC: FieldChip<F>> StrictEcPoint<F, FC> {
pub fn new(x: FC::ReducedFieldPoint, y: FC::FieldPoint) -> Self {
Self { x, y, _marker: PhantomData }
}
}
```

StrictEcPoint의 장점:

- x 좌표가 `[0, p)` 범위로 제약되어 있음
- x 좌표 동등성 검사가 더 효율적
- 점이 서로 다름을 보장할 때 유용

EcPoint로 변환:

```
impl<F: BigPrimeField, FC: FieldChip<F>> From<StrictEcPoint<F, FC>>
for EcPoint<F, FC::FieldPoint>
{
fn  from(value: StrictEcPoint<F, FC>) -> Self {
Self::new(value.x.into(), value.y)
}
}
```

ComparableEcPoint 열거형

`ComparableEcPoint` 는 x 좌표가 축약되었는지 여부를 추적하는 열거형입니다. 이를 통해 최적화된 비교 연산을 선택적으로 사용할 수 있습니다.

```
#[derive(Clone, Debug)]
pub  enum ComparableEcPoint<F: BigPrimeField, FC: FieldChip<F>> {
Strict(StrictEcPoint<F, FC>), // x가 축약된 점
NonStrict(EcPoint<F, FC::FieldPoint>), // 일반 점
}
```

From 트레이트 구현:

```
// StrictEcPoint → ComparableEcPoint
impl<F: BigPrimeField, FC: FieldChip<F>> From<StrictEcPoint<F, FC>>
for ComparableEcPoint<F, FC>
{
```

```rust
fn  from(pt: StrictEcPoint<F, FC>) -> Self {
Self::Strict(pt)
}
}

// EcPoint → ComparableEcPoint
impl<F: BigPrimeField, FC: FieldChip<F>> From<EcPoint<F, FC::FieldPoint>>
for ComparableEcPoint<F, FC>
{
fn  from(pt: EcPoint<F, FC::FieldPoint>) -> Self {
Self::NonStrict(pt)
}
}

// 참조 버전도 지원
impl<'a, F: BigPrimeField, FC: FieldChip<F>> From<&'a StrictEcPoint<F, FC>>
for ComparableEcPoint<F, FC>
{
fn  from(pt: &'a StrictEcPoint<F, FC>) -> Self {
Self::Strict(pt.clone())
}
}
```

ComparableEcPoint → EcPoint 변환:

```
impl<F: BigPrimeField, FC: FieldChip<F>> From<ComparableEcPoint<F, FC>>
for EcPoint<F, FC::FieldPoint>
{
fn  from(pt: ComparableEcPoint<F, FC>) -> Self {
match  pt {
ComparableEcPoint::Strict(pt) => Self::new(pt.x.into(), pt.y),
ComparableEcPoint::NonStrict(pt) => pt,
}
}
}
```

타입 변환 흐름

```
┌──────────────────┐
| EcPoint | ← 일반 점 (x, y 모두 일반 표현)
└──────────┬───────┘
|
| into_strict_point()
↓
┌──────────────────┐
| StrictEcPoint | ← x가 축약된 점
└──────────┬───────┘
```

```
|
| Into trait
↓
┌──────────────────┐
|ComparableEcPoint| ← 비교 가능한 점 (Strict 또는 NonStrict)
└──────────────────┘
```

사용 예시

1. 기본 점 생성

```rust
use halo2_ecc::ecc::EcPoint;

// 필드 원소로부터 점 생성
let  x = fp_chip.load_private(ctx, x_value);
let  y = fp_chip.load_private(ctx, y_value);
let P = EcPoint::new(x, y);

// 또는 EccChip을 통해 생성
let P = ecc_chip.load_private::<Secp256k1Affine>(ctx, (x_value, y_value));
```

2. StrictEcPoint 생성

```rust
// 일반 점을 strict 점으로 변환
```

```
let P = EcPoint::new(x, y);

let P_strict = into_strict_point(&fp_chip, ctx, P);



// 또는 직접 생성

let  x_reduced = fp_chip.enforce_less_than(ctx, x);

let P_strict = StrictEcPoint::new(x_reduced, y);
```

3. ComparableEcPoint 사용

```
// 함수가 ComparableEcPoint를 받는 경우

fn  ec_add_unequal<F, FC>(

chip: &FC,

ctx: &mut Context<F>,

P: impl Into<ComparableEcPoint<F, FC>>,

Q: impl Into<ComparableEcPoint<F, FC>>,

is_strict: bool,

) -> EcPoint<F, FC::FieldPoint> {

let P = P.into(); // 자동 변환

let Q = Q.into();

// is_strict = true이면 x 좌표 비교 최적화

if  is_strict {

match (&P, &Q) {

(ComparableEcPoint::Strict(p), ComparableEcPoint::Strict(q)) => {

// 축약된 x 좌표 직접 비교 (효율적)
```

```
let  x_is_equal = chip.is_equal_unenforced(ctx, p.x.clone(), q.x.clone());

chip.gate().assert_is_const(ctx, &x_is_equal, &F::ZERO);

}

_ => {

// 일반 비교 (축약 후 비교)

// ...

}

}

}

// ...

}


// 사용 시

let  result = ec_add_unequal(chip, ctx, P, Q, true); // P, Q 자동 변환
```

4. 무한원점 표현

halo2-ecc에서는 무한원점(point at infinity, 항등원)을 `(0, 0)` 으로 표현합니다:

```
// 무한원점 생성

let  zero = fp_chip.load_constant(ctx, Fp::ZERO);

let  identity = EcPoint::new(zero.clone(), zero);


// 점이 무한원점인지 확인
```

```
let is_identity = {
let x_is_zero = fp_chip.is_zero(ctx, &P.x);
let y_is_zero = fp_chip.is_zero(ctx, &P.y);
fp_chip.gate().and(ctx, x_is_zero, y_is_zero)
};
```

주의사항:

- 실제 타원곡선에서 `(0, 0)` 이 곡선 위의 점일 수 있으므로, 이 표현은 `(0, 0)` 이 곡선 위에 없다는 가정 하에 작동합니다.
- 대부분의 암호학적 곡선(secp256k1, BN254 등)에서 이 가정은 성립합니다.

타입 별칭

편의를 위한 타입 별칭:

```
// 특정 곡선에 대한 EccChip 타입
pub type BaseFieldEccChip<'chip, C> = EccChip<
'chip,
<C as CurveAffine>::ScalarExt,
FpChip<'chip, <C as CurveAffine>::ScalarExt, <C as CurveAffine>::Base>,
>;

// 사용 예
type Secp256k1EccChip<'a> = BaseFieldEccChip<'a, Secp256k1Affine>;
```

설계 철학

1. **제네릭 설계**: 다양한 필드 표현과 곡선에 대해 작동
2. **타입 안전성**: PhantomData를 사용하여 컴파일 타임 타입 체크
3. **최적화 가능성**: StrictEcPoint를 통해 선택적 최적화
4. **편의성**: Into 트레이트를 통한 자동 변환
5. **명확성**: 타입 시스템을 통해 제약 조건 표현

타원곡선 기본 연산

개요

halo2-ecc는 Short Weierstrass 형태의 타원곡선 $y^2 = x^3 + b$ 에 대한 기본 연산을 구현합니다. 모든 연산은 영지식증명 회로 내에서 제약 조건으로 표현되며, 효율성을 위해 최적화되어 있습니다.

점 덧셈 (ec_add_unequal)

소스: `halo2-ecc/src/ecc/mod.rs`

알고리즘

두 점 $P = (x_1, y_1)$과 $Q = (x_2, y_2)$가 주어졌을 때, $P + Q = (x_3, y_3)$를 계산합니다.

수학적 공식:

$$\lambda = (y_2 - y_1) / (x_2 - x_1)$$

$$x_3 = \lambda^2 - x_1 - x_2 \pmod p$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod p$$

가정:

- $P \neq Q$ ($x_1 \neq x_2$)
- P와 Q 모두 무한원점이 아님

구현

```
pub fn ec_add_unequal<F: BigPrimeField, FC: FieldChip<F>>(
chip: &FC,
ctx: &mut Context<F>,
P: impl Into<ComparableEcPoint<F, FC>>,
Q: impl Into<ComparableEcPoint<F, FC>>,
is_strict: bool,
) -> EcPoint<F, FC::FieldPoint> {
// 1. is_strict = true이면 P.x ≠ Q.x 제약 추가
let (P, Q) = check_points_are_unequal(chip, ctx, P, Q, is_strict);

// 2. λ = (y₂ - y₁) / (x₂ - x₁) 계산
let dx = chip.sub_no_carry(ctx, &Q.x, &P.x);
let dy = chip.sub_no_carry(ctx, Q.y, &P.y);
let lambda = chip.divide_unsafe(ctx, dy, dx);

// 3. x₃ = λ² - x₁ - x₂ (mod p)
let lambda_sq = chip.mul_no_carry(ctx, &lambda, &lambda);
let lambda_sq_minus_px = chip.sub_no_carry(ctx, lambda_sq, &P.x);
let x_3_no_carry = chip.sub_no_carry(ctx, lambda_sq_minus_px, Q.x);
let x_3 = chip.carry_mod(ctx, x_3_no_carry);
```

```
// 4. y₃ = λ(x₁ - x₃) - y₁ (mod p)
let dx_13 = chip.sub_no_carry(ctx, P.x, &x_3);
let lambda_dx_13 = chip.mul_no_carry(ctx, lambda, dx_13);
let y_3_no_carry = chip.sub_no_carry(ctx, lambda_dx_13, P.y);
let y_3 = chip.carry_mod(ctx, y_3_no_carry);


EcPoint::new(x_3, y_3)
}
```

제약 조건 생성 과정

1. **x 좌표 차이 계산**: $dx = x_2 - x_1$
   - `sub_no_carry` 는 carry 없이 뺄셈 수행
   - 결과는 `UnsafeFieldPoint` (오버플로우 가능)
2. **y 좌표 차이 계산**: $dy = y_2 - y_1$
3. **기울기 계산**: $\lambda = dy / dx$
   - `divide_unsafe` 는 역원을 계산하여 곱셈으로 변환
   - 제약 조건: $\lambda * dx = dy \pmod p$
4. **새 x 좌표 계산**: $x_3 = \lambda^2 - x_1 - x_2$
   - `mul_no_carry` : λ^2 계산 (carry 없음)
   - `sub_no_carry` : 연속 뺄셈
   - `carry_mod` : 최종 carry 및 modular reduction
5. **새 y 좌표 계산**: $y_3 = \lambda(x_1 - x_3) - y_1$
   - 유사한 과정으로 계산

check_points_are_unequal 함수

```
fn check_points_are_unequal<F: BigPrimeField, FC: FieldChip<F>>(
chip: &FC,
ctx: &mut Context<F>,
P: impl Into<ComparableEcPoint<F, FC>>,
Q: impl Into<ComparableEcPoint<F, FC>>,
do_check: bool,
) -> (EcPoint<F, FC::FieldPoint>, EcPoint<F, FC::FieldPoint>) {
let P = P.into();
let Q = Q.into();
if do_check {
// P.x ≠ Q.x 제약
let [x1, x2] = [&P, &Q].map(|pt| match pt {
ComparableEcPoint::Strict(pt) => pt.x.clone(),
ComparableEcPoint::NonStrict(pt) => chip.enforce_less_than(ctx, pt.x.clone()),
});
let x_is_equal = chip.is_equal_unenforced(ctx, x1, x2);
chip.gate().assert_is_const(ctx, &x_is_equal, &F::ZERO);
}
(EcPoint::from(P), EcPoint::from(Q))
}
```

역할:

- `do_check = true` : x 좌표가 다름을 제약으로 추가
- `do_check = false` : 외부 논리로 이미 보장된 경우 (수학적 정리 등)

사용 예시

```
// 두 점 생성
let P = ecc_chip.load_private::<Secp256k1Affine>(ctx, (x1, y1));
let Q = ecc_chip.load_private::<Secp256k1Affine>(ctx, (x2, y2));

// 점 덧셈 (strict 모드: P.x ≠ Q.x 제약 추가)
let R = ecc_chip.add_unequal(ctx, P, Q, true);

// 또는 직접 함수 호출
let R = ec_add_unequal(&fp_chip, ctx, P, Q, true);
```

점 뺄셈 (ec_sub_unequal)

점 뺄셈 P - Q는 P + (-Q)로 계산됩니다. -Q = $(x_2, -y_2)$입니다.

수학적 공식:

$$\lambda = -(y_2 + y_1) / (x_2 - x_1)$$

$$x_3 = \lambda^2 - x_1 - x_2 \pmod p$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod p$$

```
pub fn ec_sub_unequal<F: BigPrimeField, FC: FieldChip<F>>(
```

```
chip: &FC,

ctx: &mut Context<F>,

P: impl Into<ComparableEcPoint<F, FC>>,

Q: impl Into<ComparableEcPoint<F, FC>>,

is_strict: bool,

) -> EcPoint<F, FC::FieldPoint> {

let (P, Q) = check_points_are_unequal(chip, ctx, P, Q, is_strict);



let  dx = chip.sub_no_carry(ctx, &Q.x, &P.x);

let  sy = chip.add_no_carry(ctx, Q.y, &P.y); // y₂  + y₁



let  lambda = chip.neg_divide_unsafe(ctx, sy, dx); // -(y₂  + y₁ ) / (x₂  - x₁ )



// 나머지는 ec_add_unequal과 동일

let  lambda_sq = chip.mul_no_carry(ctx, &lambda, &lambda);

let  lambda_sq_minus_px = chip.sub_no_carry(ctx, lambda_sq, &P.x);

let  x_3_no_carry = chip.sub_no_carry(ctx, lambda_sq_minus_px, Q.x);

let  x_3 = chip.carry_mod(ctx, x_3_no_carry);



let  dx_13 = chip.sub_no_carry(ctx, P.x, &x_3);

let  lambda_dx_13 = chip.mul_no_carry(ctx, lambda, dx_13);

let  y_3_no_carry = chip.sub_no_carry(ctx, lambda_dx_13, P.y);

let  y_3 = chip.carry_mod(ctx, y_3_no_carry);
```

```
EcPoint::new(x_3, y_3)

}
```

점 배가 (ec_double)

소스: `halo2-ecc/src/ecc/mod.rs`

알고리즘

점 P = (x, y)에 대해 2P = (x_3, y_3)를 계산합니다.

수학적 공식:

$$\lambda = 3x^2 / (2y)$$

$$x_3 = \lambda^2 - 2x \pmod p$$

$$y_3 = \lambda(x - x_3) - y \pmod p$$

가정:

- y ≠ 0 (그렇지 않으면 2P = O, 무한원점)
- P는 무한원점이 아님

구현

```
pub fn ec_double<F: BigPrimeField, FC: FieldChip<F>>(

chip: &FC,

ctx: &mut Context<F>,

P: impl Into<EcPoint<F, FC::FieldPoint>>,
```

```
) -> EcPoint<F, FC::FieldPoint> {

let P = P.into();

// 1. λ = 3x² / (2y) 계산

let  two_y = chip.scalar_mul_no_carry(ctx, &P.y, 2);

let  three_x = chip.scalar_mul_no_carry(ctx, &P.x, 3);

let  three_x_sq = chip.mul_no_carry(ctx, three_x, &P.x);

let  lambda = chip.divide_unsafe(ctx, three_x_sq, two_y);


// 2. x₃  = λ² - 2x (mod p)

let  lambda_sq = chip.mul_no_carry(ctx, &lambda, &lambda);

let  two_x = chip.scalar_mul_no_carry(ctx, &P.x, 2);

let  x_3_no_carry = chip.sub_no_carry(ctx, lambda_sq, two_x);

let  x_3 = chip.carry_mod(ctx, x_3_no_carry);


// 3. y₃  = λ(x - x₃ ) - y (mod p)

let  dx = chip.sub_no_carry(ctx, P.x, &x_3);

let  lambda_dx = chip.mul_no_carry(ctx, lambda, dx);

let  y_3_no_carry = chip.sub_no_carry(ctx, lambda_dx, P.y);

let  y_3 = chip.carry_mod(ctx, y_3_no_carry);


EcPoint::new(x_3, y_3)

}
```

제약 조건 생성 과정

1. **분자 계산**: `3x²`

- `scalar_mul_no_carry(x, 3)` : 3x 계산
- `mul_no_carry(3x, x)` : $3x^2$ 계산

2. **분모 계산**: `2y`

- `scalar_mul_no_carry(y, 2)` : 2y 계산

3. **기울기 계산**: `λ = 3x² / 2y`

- 제약 조건: `λ * 2y = 3x² (mod p)`

4. **새 좌표 계산**: 점 덧셈과 유사한 과정

사용 예시

```
// 점 배가
let P = ecc_chip.load_private::<Secp256k1Affine>(ctx, (x, y));
let P2 = ecc_chip.double(ctx, P);

// 여러 번 배가
let P4 = ecc_chip.double(ctx, P2);
let P8 = ecc_chip.double(ctx, P4);
```

점 배가 후 덧셈 (ec_double_and_add_unequal)

최적화된 연산: 2P + Q를 한 번에 계산하여 중간 y 좌표 계산을 생략합니다.

알고리즘 (Montgomery ladder):

$\lambda_0 = (y_1 - y_0) / (x_1 - x_0)$

$x_2 = \lambda_0^2 - x_0 - x_1$

$\lambda_1 = \lambda_0 + 2y_0 / (x_2 - x_0)$

$x_res = \lambda_1^2 - x_0 - x_2$

$y_res = \lambda_1 (x_res - x_0) - y_0$

```
pub fn ec_double_and_add_unequal<F: BigPrimeField, FC: FieldChip<F>>(
chip: &FC,
ctx: &mut Context<F>,
P: impl Into<ComparableEcPoint<F, FC>>,
Q: impl Into<ComparableEcPoint<F, FC>>,
is_strict: bool,
) -> EcPoint<F, FC::FieldPoint> {
let P = P.into();
let Q = Q.into();
// P.x ≠ Q.x 확인
let mut x_0 = None;
if is_strict {
let [x0, x1] = [&P, &Q].map(|pt| match pt {
ComparableEcPoint::Strict(pt) => pt.x.clone(),
ComparableEcPoint::NonStrict(pt) => chip.enforce_less_than(ctx, pt.x.clone()),
});
```

```
let  x_is_equal = chip.is_equal_unenforced(ctx, x0.clone(), x1);

chip.gate().assert_is_const(ctx, &x_is_equal, &F::ZERO);

x_0 = Some(x0);

}

let P = EcPoint::from(P);

let Q = EcPoint::from(Q);


// λ₀ = (y₁  - y₀) / (x₁  - x₀)

let  dx = chip.sub_no_carry(ctx, &Q.x, &P.x);

let  dy = chip.sub_no_carry(ctx, Q.y, &P.y);

let  lambda_0 = chip.divide_unsafe(ctx, dy, dx);


// x₂  = λ₀² - x₀ - x₁

let  lambda_0_sq = chip.mul_no_carry(ctx, &lambda_0, &lambda_0);

let  lambda_0_sq_minus_x_0 = chip.sub_no_carry(ctx, lambda_0_sq, &P.x);

let  x_2_no_carry = chip.sub_no_carry(ctx, lambda_0_sq_minus_x_0, Q.x);

let  x_2 = chip.carry_mod(ctx, x_2_no_carry);


if  is_strict {

let  x_2 = chip.enforce_less_than(ctx, x_2.clone());

// x₂  ≠ x₀ 확인

let  x_is_equal = chip.is_equal_unenforced(ctx, x_0.unwrap(), x_2);

chip.range().gate().assert_is_const(ctx, &x_is_equal, &F::ZERO);
```

```
}

// λ₁  = λ₀ + 2y₀ / (x₂  - x₀)

let  two_y_0 = chip.scalar_mul_no_carry(ctx, &P.y, 2);

let  x_2_minus_x_0 = chip.sub_no_carry(ctx, &x_2, &P.x);

let  lambda_1_minus_lambda_0 = chip.divide_unsafe(ctx, two_y_0, x_2_minus_x_0);

let  lambda_1_no_carry = chip.add_no_carry(ctx, lambda_0, lambda_1_minus_lambda_


// x_res = λ₁ ² - x₀ - x₂

let  lambda_1_sq_nc = chip.mul_no_carry(ctx, &lambda_1_no_carry, &lambda_1_no_ca

let  lambda_1_sq_minus_x_0 = chip.sub_no_carry(ctx, lambda_1_sq_nc, &P.x);

let  x_res_no_carry = chip.sub_no_carry(ctx, lambda_1_sq_minus_x_0, x_2);

let  x_res = chip.carry_mod(ctx, x_res_no_carry);


// y_res = λ₁ (x_res - x₀) - y₀

let  x_res_minus_x_0 = chip.sub_no_carry(ctx, &x_res, P.x);

let  lambda_1_x_res_minus_x_0 = chip.mul_no_carry(ctx, lambda_1_no_carry, x_res_

let  y_res_no_carry = chip.sub_no_carry(ctx, lambda_1_x_res_minus_x_0, P.y);

let  y_res = chip.carry_mod(ctx, y_res_no_carry);


EcPoint::new(x_res, y_res)

}
```

장점:

- 중간 점 (P + Q)의 y 좌표를 계산하지 않음
- 제약 조건 수 감소
- 스칼라 곱셈에서 유용

점 선택 (ec_select)

조건에 따라 두 점 중 하나를 선택합니다.

```
pub fn ec_select<F: BigPrimeField, FC>(
chip: &FC,
ctx: &mut Context<F>,
P: EcPoint<F, FC::FieldPoint>,
Q: EcPoint<F, FC::FieldPoint>,
sel: AssignedValue<F>,
) -> EcPoint<F, FC::FieldPoint>
where
FC: FieldChip<F> + Selectable<F, FC::FieldPoint>,
{
let Rx = chip.select(ctx, P.x, Q.x, sel);
let Ry = chip.select(ctx, P.y, Q.y, sel);
EcPoint::new(Rx, Ry)
}
```

동작:

- `sel = 1` : P 반환

- `sel = 0`: Q 반환
- 제약 조건: `result = sel * P + (1 - sel) * Q`

곡선 위의 점 검증 (check_is_on_curve)

점이 타원곡선 위에 있는지 확인합니다.

```
pub fn check_is_on_curve<F, FC, C>(
chip: &FC,
ctx: &mut Context<F>,
P: &EcPoint<F, FC::FieldPoint>
)
where
F: BigPrimeField,
FC: FieldChip<F>,
C: CurveAffine<Base = FC::FieldType>,
{
// y² = x³ + b 확인
let lhs = chip.mul_no_carry(ctx, &P.y, &P.y); // y²
let mut rhs = chip.mul(ctx, &P.x, &P.x).into(); // x²
rhs = chip.mul_no_carry(ctx, rhs, &P.x); // x³
rhs = chip.add_constant_no_carry(ctx, rhs, C::b()); // x³ + b


let diff = chip.sub_no_carry(ctx, lhs, rhs);
chip.check_carry_mod_to_zero(ctx, diff); // diff = 0 (mod p) 제약
```

```
}
```

종합 예시

```
use halo2_ecc::ecc::{EccChip, ec_add_unequal, ec_double};
use halo2_ecc::secp256k1::Secp256k1Affine;

// EccChip 생성
let  ecc_chip = EccChip::new(&fp_chip);

// 점 로드 및 곡선 위 검증
let P = ecc_chip.load_private::<Secp256k1Affine>(ctx, (x1, y1));
let Q = ecc_chip.load_private::<Secp256k1Affine>(ctx, (x2, y2));

// 기본 연산
let P2 = ecc_chip.double(ctx, &P); // 2P
let R = ecc_chip.add_unequal(ctx, P, Q, true); // P + Q
let S = ecc_chip.sub_unequal(ctx, P, Q, true); // P - Q

// 복합 연산
let T = ecc_chip.double(ctx, &P);
let U = ecc_chip.add_unequal(ctx, T, Q, true); // 2P + Q
```

```
// 또는 최적화된 버전
let U_opt = ec_double_and_add_unequal(&fp_chip, ctx, P, Q, true);


// 조건부 선택
let  is_positive = ctx.load_witness(F::ONE);
let  result = ec_select(&fp_chip, ctx, P, Q, is_positive);
```

스칼라 곱셈 및 MSM

개요

스칼라 곱셈(scalar multiplication)은 타원곡선 암호의 핵심 연산으로, 점 P와 정수 k가 주어졌을 때 kP를 계산합니다. halo2-ecc는 다양한 최적화 기법을 사용하여 효율적인 스칼라 곱셈을 구현합니다.

스칼라 곱셈 (scalar_multiply)

소스: `halo2-ecc/src/ecc/mod.rs`

윈도우 방법 (Window Method)

스칼라를 작은 윈도우로 나누어 미리 계산된 점들을 사용하는 방법입니다.

알고리즘 개요:

1. 스칼라 k를 이진수로 변환
2. w비트 윈도우로 분할
3. 각 윈도우에 대해 미리 계산된 점 선택
4. 선택된 점들을 누적하여 최종 결과 계산

함수 시그니처

```rust
pub fn scalar_multiply<F: BigPrimeField, FC, C>(
chip: &FC,
ctx: &mut Context<F>,
P: EcPoint<F, FC::FieldPoint>,
scalar: Vec<AssignedValue<F>>,
max_bits: usize,
window_bits: usize,
) -> EcPoint<F, FC::FieldPoint>
where
FC: FieldChip<F> + Selectable<F, FC::FieldPoint>,
C: CurveAffineExt<Base = FC::FieldType>,
```

파라미터:

- **P**: 기저 점
- **scalar**: 스칼라 값 (여러 AssignedValue로 표현, 큰 정수 지원)
- **max_bits**: 각 scalar 원소의 최대 비트 수
- **window_bits**: 윈도우 크기 (일반적으로 4)

가정:

- `window_bits != 0`
- P의 차수가 최소 `2^{window_bits}` 이상
- 곡선에 차수 2인 점이 없음
- `scalar_i < 2^{max_bits}`
- `max_bits <= modulus::<F>.bits()`

구현 세부사항

```
// 1. 스칼라를 비트로 변환
let total_bits = max_bits * scalar.len();
let num_windows = (total_bits + window_bits - 1) / window_bits;
let rounded_bitlen = num_windows * window_bits;

let mut bits = Vec::with_capacity(rounded_bitlen);
for x in scalar {
let mut new_bits = chip.gate().num_to_bits(ctx, x, max_bits);
bits.append(&mut new_bits);
}
// 0으로 패딩하여 윈도우 크기의 배수로 만듦
let zero_cell = ctx.load_zero();
bits.resize(rounded_bitlen, zero_cell);

// 2. 미리 계산된 점들 생성
// cached_points[i] = i * P for i in 0..2^window_bits
let cache_size = 1usize << window_bits;
let any_point = load_random_point::<F, FC, C>(chip, ctx);
let mut cached_points = Vec::with_capacity(cache_size);
cached_points.push(any_point); // 더미 점 (0 * P 대신)
cached_points.push(P.clone()); // 1 * P
```

```
for idx in 2..cache_size {
if idx == 2 {
let double = ec_double(chip, ctx, &P);
cached_points.push(double);
} else {
let new_point = ec_add_unequal(chip, ctx, &cached_points[idx - 1], &P, false);
cached_points.push(new_point);
}
}

// 3. 윈도우별로 점 선택 및 누적
let mut curr_point = ec_select_from_bits(
chip,
ctx,
&cached_points,
&bits[rounded_bitlen - window_bits..rounded_bitlen],
);

for idx in 1..num_windows {
// 현재 점을 window_bits만큼 배가
let mut mult_point = curr_point.clone();
for _ in 0..window_bits {
mult_point = ec_double(chip, ctx, mult_point);
```

```
}

// 다음 윈도우에 해당하는 점 선택

let  add_point = ec_select_from_bits(

chip,

ctx,

&cached_points,

&bits[rounded_bitlen - window_bits * (idx + 1)..rounded_bitlen - window_bits * i

);

// 점 덧셈

let  mult_and_add = ec_add_unequal(chip, ctx, &mult_point, &add_point, true);

// 조건부 선택 (윈도우가 0인 경우 처리)

curr_point = /* ... */;

}
```

사용 예시

```
use halo2_ecc::ecc::{EccChip, scalar_multiply};

use halo2_ecc::secp256k1::{Secp256k1Affine, Fq};



// 스칼라 값 로드 (256비트를 88비트 limb 3개로 표현)

let  scalar_val = BigUint::from_str("12345678...").unwrap();

let  scalar_limbs = decompose_biguint(&scalar_val, 3, 88);

let  scalar = scalar_limbs

.into_iter()
```

```
.map(|limb| ctx.load_witness(limb))
.collect::<Vec<_>>();

// 기저 점
let P = ecc_chip.load_private::<Secp256k1Affine>(ctx, (x, y));

// 스칼라 곱셈: k * P
let  result = scalar_multiply::<F, _, Secp256k1Affine>(
&fp_chip,
ctx,
P,
scalar,
88, // max_bits
4, // window_bits
);

// 또는 EccChip 메서드 사용
let  result = ecc_chip.scalar_mult::<Secp256k1Affine>(
ctx,
P,
scalar,
88,
4,
```

```
);
```

Multi-Scalar Multiplication (MSM)

MSM은 여러 스칼라 곱셈을 동시에 계산하는 연산입니다:

$$result = k_1 P_1 + k_2 P_2 + ... + k_n P_n$$

multi_scalar_multiply 함수

소스: `halo2-ecc/src/ecc/mod.rs`

```rust
pub fn multi_scalar_multiply<F: BigPrimeField, FC, C>(
chip: &FC,
ctx: &mut Context<F>,
P: &[EcPoint<F, FC::FieldPoint>],
scalars: Vec<Vec<AssignedValue<F>>>,
max_bits: usize,
window_bits: usize,
) -> EcPoint<F, FC::FieldPoint>
where
FC: FieldChip<F> + Selectable<F, FC::FieldPoint>,
C: CurveAffineExt<Base = FC::FieldType>,
```

파라미터:

- **P**: 점들의 배열

- **scalars**: 각 점에 대응하는 스칼라들
- **max_bits**: 스칼라 limb당 최대 비트 수
- **window_bits**: 윈도우 크기

가정:

- `points.len() == scalars.len()`
- `scalars[i].len() == scalars[j].len()` (모든 i, j)
- 모든 점이 곡선 위에 있거나 무한원점
- 점 `(0, 0)` 은 무한원점을 나타냄
- y 좌표가 0인 유일한 점은 무한원점

Simultaneous 2^w-Ary Method

MSM은 Simultaneous 2^w-Ary Method를 사용하여 최적화됩니다.

알고리즘:

1. 모든 스칼라를 비트로 변환하고 윈도우로 분할
2. 각 점에 대해 미리 계산된 점들 생성
3. 랜덤 누적 점(random accumulation point) 사용하여 충돌 방지
4. 윈도우별로 모든 점의 기여도를 동시에 계산

```
// 1. 스칼라를 비트로 변환
let  rounded_bits = scalars
.into_iter()
.flat_map(|scalar| {
scalar
.into_iter()
```

```
.flat_map(|scalar_chunk| chip.gate().num_to_bits(ctx, scalar_chunk, max_bits))
.chain(std::iter::repeat(zero_cell).take(rounded_bitlen - total_bits))
.collect_vec()
})
.collect_vec();


// 2. 랜덤 기저 점 생성 (충돌 방지용)
let base = load_random_point::<F, FC, C>(chip, ctx);
let mut rand_start_vec = Vec::with_capacity(k + window_bits);
rand_start_vec.push(base);
for idx in 1..(k + window_bits) {
let base_mult = ec_double(chip, ctx, &rand_start_vec[idx - 1]);
rand_start_vec.push(base_mult);
}


// 3. 각 점에 대해 캐시 생성
// cached_points[i][j] = (1 - 2^w) * 2^i * A + j * P_i
let cache_size = 1usize << window_bits;
let mut cached_points = Vec::with_capacity(k * cache_size);
for (idx, point) in P.iter().enumerate() {
let is_infinity = chip.is_zero(ctx, &point.y);
let neg_mult_rand_start = ec_sub_unequal(
chip,
```

```
ctx,
&rand_start_vec[idx],
&rand_start_vec[idx + window_bits],
true,
);
let point = into_strict_point(chip, ctx, point.clone());
let neg_mult_rand_start = into_strict_point(chip, ctx, neg_mult_rand_start);
cached_points.push(neg_mult_rand_start);
for _ in 0..(cache_size - 1) {
let prev = cached_points.last().unwrap().clone();
let mut new_point = ec_add_unequal(chip, ctx, &prev, &point, true);
new_point = ec_select(chip, ctx, prev.into(), new_point, is_infinity);
let new_point = into_strict_point(chip, ctx, new_point);
cached_points.push(new_point);
}
}

// 4. 초기 점: (2^{k+1} - 1) * A
let start_point = ec_sub_unequal(
chip,
ctx,
&rand_start_vec[k],
&rand_start_vec[0],
```

```
true,
);
let mut curr_point = start_point.clone();


// 5. 윈도우별로 누적
for idx in 0..num_windows {
// window_bits만큼 배가
for _ in 0..window_bits {
curr_point = ec_double(chip, ctx, curr_point);
}
// 각 점의 기여도 추가
for (cached_points, rounded_bits) in
cached_points.chunks(cache_size).zip(rounded_bits.chunks(rounded_bitlen))
{
let add_point = ec_select_from_bits(
chip,
ctx,
cached_points,
&rounded_bits[rounded_bitlen - window_bits * (idx + 1)..rounded_bitlen - window_
);
curr_point = ec_add_unequal(chip, ctx, curr_point, add_point, true);
}
}
```

```
// 6. 랜덤 점 제거
ec_sub_strict(chip, ctx, curr_point, start_point)
```

Pippenger 알고리즘

소스: `halo2-ecc/src/ecc/pippenger.rs`

Pippenger 알고리즘은 대규모 MSM을 위한 최적화된 알고리즘입니다. 점의 개수가 많을 때 (일반적으로 > 25) 더 효율적입니다.

핵심 아이디어

1. **Bucket Method**: 스칼라 값에 따라 점들을 버킷으로 그룹화
2. **Clumping**: 여러 점을 한 번에 처리하여 제약 조건 수 감소
3. **병렬 처리**: rayon을 사용한 witness 생성 병렬화

multi_exp_par 함수

```
pub fn multi_exp_par<F: BigPrimeField, FC, C>(
chip: &FC,
builder: &mut SinglePhaseCoreManager<F>,
points: &[EcPoint<F, FC::FieldPoint>],
scalars: Vec<Vec<AssignedValue<F>>>,
max_bits: usize,
clump_factor: usize,
) -> EcPoint<F, FC::FieldPoint>
```

```
where
FC: FieldChip<F> + Selectable<F, FC::FieldPoint> + Selectable<F, FC::ReducedFiel
C: CurveAffineExt<Base = FC::FieldType>,
```

파라미터:

- **clump_factor**: 한 번에 처리할 점의 개수 (일반적으로 4)

알고리즘:

```
let c = clump_factor;

// 랜덤 기저 점
let any_base = load_random_point::<F, FC, C>(chip, ctx);
let mut acc = Vec::with_capacity(bool_scalars.len());
let mut bucket = Vec::with_capacity(1 << c);
let mut any_point = any_base.clone();

for (round, points_clump) in points.chunks(c).enumerate() {
// 라운드마다 다른 랜덤 점 사용
if round > 0 {
any_point = ec_double(chip, ctx, any_point);
}
// 버킷 생성: 모든 가능한 조합
// bucket[j] = rand_point + sum_{i: bit[i]=1} points[i]
bucket.clear();
```

```
let strict_any_point = into_strict_point(chip, ctx, any_point.clone());
bucket.push(strict_any_point);
for (i, point) in points_clump.iter().enumerate() {
let is_infinity = chip.is_zero(ctx, &point.y);
let point = into_strict_point(chip, ctx, point.clone());
for j in 0..(1 << i) {
let mut new_point = ec_add_unequal(chip, ctx, &bucket[j], &point, true);
new_point = ec_select(chip, ctx, (&bucket[j]).into(), new_point, is_infinity);
let new_point = into_strict_point(chip, ctx, new_point);
bucket.push(new_point);
}
}
// 비트에 따라 버킷에서 선택
for (j, bits) in bool_scalars.iter().enumerate() {
let multi_prod = strict_ec_select_from_bits(
chip,
ctx,
&bucket,
&bits[round * c..round * c + points_clump.len()],
);
if round == 0 {
acc.push(multi_prod);
} else {
```

```
let _acc = ec_add_unequal(chip, ctx, &acc[j], multi_prod, true);
acc[j] = into_strict_point(chip, ctx, _acc);
}
}
}

// 랜덤 점 제거
any_point = ec_double(chip, ctx, any_point);
any_point = ec_sub_unequal(chip, ctx, any_point, any_base, false);
```

고정 기저점 스칼라 곱셈 (Fixed Base)

소스: `halo2-ecc/src/ecc/fixed_base.rs`

기저 점이 상수(constant)인 경우, 미리 계산을 통해 더욱 최적화할 수 있습니다.

scalar_multiply (fixed base)

```
pub fn scalar_multiply<F, FC, C>(
chip: &FC,
ctx: &mut Context<F>,
point: &C, // 상수 점
scalar: Vec<AssignedValue<F>>,
max_bits: usize,
window_bits: usize,
```

```
) -> EcPoint<F, FC::FieldPoint>
where
F: BigPrimeField,
C: CurveAffineExt,
FC: FieldChip<F, FieldType = C::Base> + Selectable<F, FC::FieldPoint>,
```

최적화:

1. **Witness 생성 시 미리 계산**: 모든 캐시 점을 Jacobian 좌표로 계산
2. **Batch Normalization**: 한 번에 모든 점을 affine 좌표로 변환 (역원 계산 최소화)
3. **상수로 로드**: 미리 계산된 점들을 상수로 회로에 로드

```
// 1. Jacobian 좌표로 캐시 점 계산
let base_pt = point.to_curve();
let mut increment = base_pt;
let cached_points_jacobian = (0..num_windows)
.flat_map(|i| {
let mut curr = increment;
let cache_vec = std::iter::once(increment)
.chain((1..(1usize << min(window_bits, total_bits - i * window_bits))).map(|_| {
let prev = curr;
curr += increment;
prev
}))
.collect::<Vec<_>>();
```

```
increment = curr;

cache_vec

})

.collect::<Vec<_>>();


// 2. Batch normalization (매우 효율적)

let mut cached_points_affine = vec![C::default(); cached_points_jacobian.len()

C::Curve::batch_normalize(&cached_points_jacobian, &mut cached_points_affine);


// 3. 상수로 로드

let cached_points = cached_points_affine

.into_iter()

.map(|point| {

let (x, y) = point.into_coordinates();

let [x, y] = [x, y].map(|x| chip.load_constant(ctx, x));

EcPoint::new(x, y)

})

.collect_vec();


// 4. 윈도우 방법으로 계산 (variable base와 유사)

// ...
```

fixed_base_msm_par

고정 기저점 MSM의 병렬 버전:

```
pub fn msm_par<F, FC, C>(
chip: &EccChip<F, FC>,
builder: &mut SinglePhaseCoreManager<F>,
points: &[C], // 상수 점들
scalars: Vec<Vec<AssignedValue<F>>>,
max_scalar_bits_per_cell: usize,
window_bits: usize,
) -> EcPoint<F, FC::FieldPoint>
```

최적화:

- 모든 점의 캐시를 한 번에 batch normalize
- rayon을 사용한 병렬 witness 생성
- 각 스칼라 곱셈을 별도 스레드에서 계산

variable_base_msm vs fixed_base_msm

EccChip 메서드:

```
impl<'chip, F: BigPrimeField, FC: FieldChip<F>> EccChip<'chip, F, FC> {
// Variable base MSM (점이 witness)
pub fn variable_base_msm<C>(
&self,
thread_pool: &mut SinglePhaseCoreManager<F>,
P: &[EcPoint<F, FC::FieldPoint>],
```

```
scalars: Vec<Vec<AssignedValue<F>>>,
max_bits: usize,
) -> EcPoint<F, FC::FieldPoint>
where
C: CurveAffineExt<Base = FC::FieldType>,
FC: Selectable<F, FC::ReducedFieldPoint>,
{
// window_bits = 4가 경험적으로 최적
self.variable_base_msm_custom::<C>(thread_pool, P, scalars, max_bits, 4)
}

// Fixed base MSM (점이 상수)
pub fn fixed_base_msm<C>(
&self,
builder: &mut SinglePhaseCoreManager<F>,
points: &[C],
scalars: Vec<Vec<AssignedValue<F>>>,
max_scalar_bits_per_cell: usize,
) -> EcPoint<F, FC::FieldPoint>
where
C: CurveAffineExt,
FC: FieldChip<F, FieldType = C::Base> + Selectable<F, FC::FieldPoint>,
{
```

```
self.fixed_base_msm_custom::<C>(builder, points, scalars, max_scalar_bits_per_ce
}
}
```

성능 비교

방법	점 개수	최적 window_bits	특징
scalar_multiply	1	4	단일 스칼라 곱셈
multi_scalar_multiply	≤ 25	4	작은 MSM
Pippenger (variable)	> 25	4	대규모 MSM, 병렬 처리
fixed_base (single)	1	4	고정 기저점, batch normalize
fixed_base_msm	any	4	고정 기저점 MSM, 가장 효율적

사용 예시

```
// 1. 단일 스칼라 곱셈
let  result = ecc_chip.scalar_mult::<Secp256k1Affine>(
ctx,
P,
scalar,
88, // max_bits
4, // window_bits
);
```

```
// 2. Variable base MSM
let points = vec![P1, P2, P3];
let scalars = vec![k1, k2, k3];
let result = ecc_chip.variable_base_msm::<Secp256k1Affine>(
builder,
&points,
scalars,
88,
);


// 3. Fixed base MSM (가장 효율적)
let generator = Secp256k1Affine::generator();
let points = vec![generator; 100];
let result = ecc_chip.fixed_base_msm::<Secp256k1Affine>(
builder,
&points,
scalars,
88,
);
```

최적화 기법 요약

1. **윈도우 방법**: 미리 계산으로 배가 횟수 감소

2. **Batch Normalization**: 역원 계산 최소화

3. **랜덤 누적 점**: 점 충돌 방지, strict 모드 회피

4. **병렬 처리**: witness 생성 병렬화

5. **Pippenger 알고리즘**: 대규모 MSM 최적화

6. **고정 기저점 최적화**: 상수 점 활용

필드 연산 최적화

개요

halo2-ecc의 필드 연산은 네이티브 필드보다 큰 소수체에서의 연산을 효율적으로 구현하기 위해 여러 최적화 기법을 사용합니다. 핵심은 **limb 기반 표현**과 **CRT(Chinese Remainder Theorem) 표현**을 결합하는 것입니다.

FpChip 구조

소스: `halo2-ecc/src/fields/fp.rs`

구조체 정의

```
#[derive(Clone, Debug)]
pub struct FpChip<'range, F: BigPrimeField, Fp: BigPrimeField> {
pub range: &'range RangeChip<F>,

pub limb_bits: usize, // 각 limb의 비트 수 (예: 88)
pub num_limbs: usize, // limb 개수 (예: 3)

pub num_limbs_bits: usize, // num_limbs의 비트 길이
```

```
pub num_limbs_log2_ceil: usize,
pub limb_bases: Vec<F>, // [1, 2^limb_bits, 2^(2*limb_bits), ...]
pub limb_base_big: BigInt, // 2^limb_bits
pub limb_mask: BigUint, // 2^limb_bits - 1

pub p: BigInt, // 소수체 모듈러스
pub p_limbs: Vec<F>, // p를 limb로 분해
pub p_native: F, // p mod (네이티브 필드 모듈러스)

pub native_modulus: BigUint, // 네이티브 필드 모듈러스
_marker: PhantomData<Fp>,
}
```

제네릭 파라미터:

- **F**: 네이티브 필드 (회로의 기본 필드, 예: `Fr`)
- **Fp**: 타겟 소수체 (타원곡선의 기저 필드, 예: `Fq`)

생성자

```
impl<'range, F: BigPrimeField, Fp: BigPrimeField> FpChip<'range, F, Fp> {
pub fn new(range: &'range RangeChip<F>, limb_bits: usize, num_limbs: usize) ->
assert!(limb_bits > 0);
assert!(num_limbs > 0);
assert!(limb_bits <= F::CAPACITY as usize);
```

```
let limb_mask = (BigUint::from(1u64) << limb_bits) - 1usize;
let p = modulus::<Fp>();
let p_limbs = decompose_biguint(&p, num_limbs, limb_bits);
let native_modulus = modulus::<F>();
let p_native = biguint_to_fe::<F>(&(&p % &native_modulus));


let limb_base = biguint_to_fe::<F>(&(BigUint::one() << limb_bits));
let mut limb_bases = Vec::with_capacity(num_limbs);
limb_bases.push(F::ONE);
while limb_bases.len() != num_limbs {
limb_bases.push(limb_base * limb_bases.last().unwrap());
}


Self {
range,
limb_bits,
num_limbs,
num_limbs_bits: bit_length(num_limbs as u64),
num_limbs_log2_ceil: log2_ceil(num_limbs as u64),
limb_bases,
limb_base_big: BigInt::one() << limb_bits,
limb_mask,
p: p.into(),
```

```
p_limbs,

p_native,

native_modulus,

_marker: PhantomData,

}

}

}
```

Limb 기반 표현

큰 정수를 여러 작은 조각(limb)으로 분할하여 표현합니다.

개념

```
value = limb[0] + limb[1] * 2^n + limb[2] * 2^(2n) + ... + limb[k-1] * 2^((k-1)n
```

여기서:

- `n = limb_bits` (예: 88비트)
- `k = num_limbs` (예: 3개)

예시: secp256k1의 Fq 표현

```
Fq 모듈러스 p = 2^256 - 2^32 - 977 (약 256비트)

limb_bits = 88

num_limbs = 3
```

```
p를 limb로 분해:

limb[0] = p & ((1 << 88) - 1)

limb[1] = (p >> 88) & ((1 << 88) - 1)

limb[2] = (p >> 176) & ((1 << 88) - 1)
```

CRT (Chinese Remainder Theorem) 표현

소스: `halo2-ecc/src/bigint/`

CRTInteger 구조체

```rust
pub struct CRTInteger<F: ScalarField> {
// Limb 표현 (오버플로우 가능)
pub truncation: OverflowInteger<F>,
// 네이티브 필드에서의 값 (mod native_modulus)
pub native: AssignedValue<F>,
// 실제 BigInt 값 (witness 생성용)
pub value: BigInt,
}

pub struct OverflowInteger<F: ScalarField> {
pub limbs: Vec<AssignedValue<F>>,
pub max_limb_bits: usize, // 각 limb의 최대 비트 수
}
```

CRT의 장점:

1. **이중 검증**: limb 표현과 네이티브 표현 모두 유지
2. **효율적인 동등성 검사**: 네이티브 값으로 빠른 비교
3. **오버플로우 추적**: `max_limb_bits` 로 오버플로우 관리

ProperCrtUint

올바르게 범위 검사된 CRT 정수:

```
pub struct ProperCrtUint<F: ScalarField>(pub CRTInteger<F>);
```

특징:

- 각 limb가 `[0, 2^limb_bits)` 범위로 제약됨
- 전체 값이 올바른 범위 내에 있음을 보장

carry 없는 연산 (no_carry operations)

중간 계산에서 carry를 지연시켜 제약 조건 수를 줄입니다.

mul_no_carry

소스: `halo2-ecc/src/bigint/mul_no_carry.rs`

```
pub fn mul_no_carry<F: ScalarField>(
gate: &impl GateInstructions<F>,
ctx: &mut Context<F>,
a: CRTInteger<F>,
b: CRTInteger<F>,
num_limbs_log2_ceil: usize,
```

```
) -> CRTInteger<F> {
let  out_limbs = a.truncation.limbs.iter().enumerate().map(|(i, a_limb)| {
gate.inner_product(
ctx,
b.truncation.limbs.iter().take(i + 1).rev().copied(),
a.truncation.limbs[..=i].iter().map(|x| Existing(*x)),
)
}).collect();


let  out_native = gate.mul(ctx, a.native, b.native);
let  out_val = &a.value * &b.value;
let  out_max_limb_bits = a.truncation.max_limb_bits
+ b.truncation.max_limb_bits
+ num_limbs_log2_ceil;


CRTInteger::new(
OverflowInteger::new(out_limbs, out_max_limb_bits),
out_native,
out_val,
)
}
```

동작:

1. 각 출력 limb는 입력 limb들의 내적으로 계산
2. 네이티브 값은 단순 곱셈
3. `max_limb_bits` 업데이트하여 오버플로우 추적
4. **carry는 수행하지 않음** - 나중에 `carry_mod` 에서 처리

add_no_carry

```
pub fn add_no_carry<F: ScalarField>(
gate: &impl GateInstructions<F>,
ctx: &mut Context<F>,
a: CRTInteger<F>,
b: CRTInteger<F>,
) -> CRTInteger<F> {
let out_limbs = a.truncation.limbs
.iter()
.zip(b.truncation.limbs.iter())
.map(|(a_limb, b_limb)| gate.add(ctx, *a_limb, *b_limb))
.collect();

let out_native = gate.add(ctx, a.native, b.native);
let out_val = &a.value + &b.value;
let out_max_limb_bits = max(a.truncation.max_limb_bits, b.truncation.max_limb_b

CRTInteger::new(
```

```
OverflowInteger::new(out_limbs, out_max_limb_bits),

out_native,

out_val,

)

}
```

sub_no_carry

```
pub fn sub_no_carry<F: ScalarField>(

gate: &impl GateInstructions<F>,

ctx: &mut Context<F>,

a: CRTInteger<F>,

b: CRTInteger<F>,

) -> CRTInteger<F> {

let out_limbs = a.truncation.limbs

.iter()

.zip(b.truncation.limbs.iter())

.map(|(a_limb, b_limb)| gate.sub(ctx, *a_limb, *b_limb))

.collect();


let out_native = gate.sub(ctx, a.native, b.native);

let out_val = &a.value - &b.value;

let out_max_limb_bits = max(a.truncation.max_limb_bits, b.truncation.max_limb_b
```

```
CRTInteger::new(
OverflowInteger::new(out_limbs, out_max_limb_bits),
out_native,
out_val,
)
}
```

carry 및 modular reduction (carry_mod)

소스: `halo2-ecc/src/bigint/carry_mod.rs`

오버플로우된 limb 표현을 올바른 범위로 정규화하고 modular reduction을 수행합니다.

알고리즘

```
pub fn carry_mod<F: ScalarField>(
range: &impl RangeInstructions<F>,
ctx: &mut Context<F>,
a: CRTInteger<F>,
num_limbs_bits: usize,
p: &BigInt,
p_limbs: &[F],
p_native: F,
limb_bits: usize,
limb_bases: &[F],
```

```
limb_base_big: &BigInt,

) -> ProperCrtUint<F> {

// 1. witness 계산: a를 p로 나눈 몫과 나머지

let  a_val = &a.value;

let (quot_val, out_val) = a_val.div_rem(p);

// 2. 몫과 나머지를 limb로 분해하여 로드

let  quot = // ... witness 할당

let  out = // ... witness 할당

// 3. 제약 조건: a = quot * p + out

// quot * p 계산 (no carry)

let  quot_p = mul_no_carry(gate, ctx, quot, p_crt, num_limbs_log2_ceil);

// quot * p + out 계산 (no carry)

let  quot_p_plus_out = add_no_carry(gate, ctx, quot_p, out.clone());

// 4. limb별로 동등성 검사

for (a_limb, qp_limb) in  a.truncation.limbs.iter().zip(quot_p_plus_out.truncati

ctx.constrain_equal(a_limb, qp_limb);

}

// 5. 네이티브 값 검사

ctx.constrain_equal(&a.native, &quot_p_plus_out.native);

// 6. 범위 검사: out < p

range_check(range, ctx, out.clone(), limb_bits);

enforce_less_than_p(range, ctx, out.clone(), p_limbs, limb_bits);

ProperCrtUint(out)
```

```
}
```

핵심 아이디어:

1. Witness로 몫(quotient)과 나머지(remainder) 계산
2. `a = quot * p + remainder` 제약 조건 생성
3. `remainder < p` 범위 검사

나눗셈 (divide_unsafe)

소스: `halo2-ecc/src/fields/mod.rs`

구현

```rust
fn divide_unsafe(
&self,
ctx: &mut Context<F>,
a: impl Into<Self::UnsafeFieldPoint>,
b: impl Into<Self::UnsafeFieldPoint>,
) -> Self::FieldPoint {
let a = a.into();
let b = b.into();
// 1. Witness 계산: a / b
let a_val = self.get_assigned_value(&a);
let b_val = self.get_assigned_value(&b);
let b_inv: Self::FieldType = Option::from(b_val.invert()).unwrap_or_default();
let quot_val = a_val * b_inv;
```

```
// 2. 몫을 witness로 로드
let quot = self.load_private(ctx, quot_val);


// 3. 제약 조건: quot * b - a = 0 (mod p)
let quot_b = self.mul_no_carry(ctx, quot.clone(), b);
let quot_constraint = self.sub_no_carry(ctx, quot_b, a);
self.check_carry_mod_to_zero(ctx, quot_constraint);


quot
}
```

동작:

1. Witness 생성 시 역원 계산 (`b.invert()`)
2. 몫 `quot = a * b^(-1)` 계산
3. 제약 조건: `quot * b = a (mod p)` 검증

"unsafe" 이유:

- `b = 0` 인지 검사하지 않음
- 호출자가 `b ≠ 0` 을 보장해야 함
- 안전한 버전 `divide` 는 `b ≠ 0` 제약 추가

배치 역원 (Batch Inversion)

여러 원소의 역원을 한 번에 계산하여 효율성을 높입니다.

Montgomery's Trick

```
// 입력: [a₁ , a₂ , ..., aₙ]
// 출력: [a₁⁻¹, a₂⁻¹, ..., aₙ⁻¹]

// 1. 누적 곱 계산
let mut products = vec![a[0]];
for i in 1..n {
products.push(products[i-1] * a[i]);
}

// 2. 마지막 누적 곱의 역원 계산 (단 한 번의 역원 연산!)
let inv_product = products[n-1].invert();

// 3. 역방향으로 개별 역원 계산
let mut inv = vec![F::ZERO; n];
inv[n-1] = inv_product * products[n-2];
for i in (0..n-1).rev() {
inv[i] = inv_product * products[i-1];
inv_product *= a[i+1];
}
```

장점:

- n개 원소의 역원을 1번의 역원 연산 + 3n번의 곱셈으로 계산

- 역원 연산은 매우 비싸므로 (수백 배) 큰 성능 향상

사용 예: Fixed Base MSM

```
// Jacobian 좌표로 계산된 점들
let cached_points_jacobian = /* ... */;

// Batch normalize: 한 번에 모든 점을 affine 좌표로 변환
let mut cached_points_affine = vec![C::default(); cached_points_jacobian.len()
C::Curve::batch_normalize(&cached_points_jacobian, &mut cached_points_affine);
```

최적화 기법 요약

1. Limb 기반 표현

- 큰 정수를 작은 조각으로 분할
- 네이티브 필드 연산 활용
- 병렬 처리 가능

2. CRT 표현

- Limb 표현 + 네이티브 표현
- 이중 검증으로 정확성 보장
- 효율적인 동등성 검사

3. Carry 지연

- 중간 계산에서 carry 생략
- 제약 조건 수 감소

- 최종 단계에서만 carry 수행

4. Batch Normalization

- 여러 역원을 한 번에 계산
- Montgomery's trick 사용
- 역원 연산 횟수 최소화

5. 범위 검사 최적화

- Lookup 테이블 활용
- 필요한 경우에만 수행
- `enforce_less_than` 으로 reduced form 보장

성능 비교

| 연산 | 제약 조건 수 (근사) | 비고 |

|------|---------------------|------|

| add_no_carry | ~3 per limb | carry 없음 |

| mul_no_carry | ~9 per limb | carry 없음 |

| carry_mod | ~20 per limb | carry + range check |

| divide_unsafe | ~30 per limb | 역원 + 곱셈 + carry |

| batch_normalize (n개) | 1 inversion | witness 생성 시 |

사용 예시

```
use halo2_ecc::fields::fp::FpChip;

use halo2_ecc::secp256k1::Fq;
```

```
// FpChip 생성
let  fp_chip = FpChip::<F, Fq>::new(
&range_chip,
88, // limb_bits
3, // num_limbs
);

// 필드 원소 로드
let  a = fp_chip.load_private(ctx, a_val);
let  b = fp_chip.load_private(ctx, b_val);

// 연산 수행
// 1. 곱셈 (carry 없음)
let  ab_no_carry = fp_chip.mul_no_carry(ctx, &a, &b);

// 2. 덧셈 (carry 없음)
let  ab_plus_a = fp_chip.add_no_carry(ctx, ab_no_carry, &a);

// 3. Carry 및 modular reduction
let  result = fp_chip.carry_mod(ctx, ab_plus_a);

// 또는 한 번에
```

```
let ab = fp_chip.mul(ctx, a, b); // mul_no_carry + carry_mod


// 나눗셈
let a_div_b = fp_chip.divide_unsafe(ctx, a, b);


// 안전한 나눗셈 (b ≠ 0 제약 추가)
let a_div_b_safe = fp_chip.divide(ctx, a, b);
```

타원곡선 연산에서의 활용

```
// 점 덧셈에서 기울기 계산: λ = (y₂ - y₁) / (x₂ - x₁)
let dx = fp_chip.sub_no_carry(ctx, &Q.x, &P.x);
let dy = fp_chip.sub_no_carry(ctx, Q.y, &P.y);
let lambda = fp_chip.divide_unsafe(ctx, dy, dx);


// x₃ = λ² - x₁ - x₂
let lambda_sq = fp_chip.mul_no_carry(ctx, &lambda, &lambda);
let lambda_sq_minus_px = fp_chip.sub_no_carry(ctx, lambda_sq, &P.x);
let x_3_no_carry = fp_chip.sub_no_carry(ctx, lambda_sq_minus_px, Q.x);
let x_3 = fp_chip.carry_mod(ctx, x_3_no_carry); // 최종 carry


// y₃ = λ(x₁ - x₃) - y₁
let dx_13 = fp_chip.sub_no_carry(ctx, P.x, &x_3);
```

```
let lambda_dx_13 = fp_chip.mul_no_carry(ctx, lambda, dx_13);

let y_3_no_carry = fp_chip.sub_no_carry(ctx, lambda_dx_13, P.y);

let y_3 = fp_chip.carry_mod(ctx, y_3_no_carry);
```

최적화 효과:

- Carry를 마지막에만 수행하여 제약 조건 수 감소
- 중간 계산은 오버플로우 허용
- 최종 결과만 올바른 범위로 정규화

다음 단계

타원곡선 암호 연산을 이해했다면, 이제 실제 암호학적 프로토콜의 구현을 학습하세요:

→ **07. 실전 예제 분석**: ECDSA 서명 검증과 BN254 Pairing의 완전한 구현

관련 문서

- 01. 개요 및 아키텍처: halo2-lib 전체 구조
- 02. 핵심 개념: Context, AssignedValue, QuantumCell
- 04. 제약 조건 시스템: FlexGate, RangeGate
- 05. 가상 영역 관리: 회로 최적화
- 07. 실전 예제 분석: ECDSA, Pairing 구현

참고 자료

소스 파일:

- `halo2-ecc/src/lib.rs` : ECC 라이브러리 진입점

- `halo2-ecc/Cargo.toml` : 의존성 및 기능 설정
- `halo2-ecc/src/ecc/mod.rs` : 타원곡선 점 연산
- `halo2-ecc/src/fields/mod.rs` : 필드 연산
- `halo2-ecc/src/bigint/` : 큰 정수 연산

외부 링크:

- Elliptic Curve Cryptography
- Pippenger's Algorithm
- Chinese Remainder Theorem